

Operational Bisimulation for Equation Identity: A Methodology for Cross-Framework Computational Equivalence

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
AIME Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

June 4, 2026

Abstract

We develop a methodology for proving that two formalisms with distinct operational semantics compute the same equation. The methodology rests on operational bisimulation: a relation between configurations of the two systems that is preserved under transitions and that respects the observable outputs. We make three contributions. (i) We give a precise definition of *equation identity*—the claim that two systems compute the same input-output relation—and establish a soundness theorem stating that bisimulation between operational semantics implies equation identity. (ii) We provide a coinductive proof principle for bisimulation that is modular: bisimulations may be exhibited as relations satisfying a one-step compatibility property, eliminating the need to enumerate runs over all inputs. (iii) We supply a decidable verification procedure based on partition refinement, with complexity $\mathcal{O}((m + n) \log n)$ in the configuration space, where m is the number of transitions and n the number of configurations. We further establish four structural results: bisimulation is the largest equivalence preserving observations (Theorem 3.4), bisimulation is preserved under sequential and parallel composition (Theorem 5.2), bisimulation supports a coinduction principle in the up-to style (Theorem 4.3), and bisimulation between deterministic systems coincides with extensional equivalence (Theorem 7.2). We illustrate the methodology with four worked examples: two operational semantics for arithmetic

expressions, two implementations of a queue, two computations of the factorial, and two finite-state automata for the same regular language. In each case, the equation identity claim is reduced to the exhibition of an explicit bisimulation, and the bisimulation is verified by a finite case analysis. The framework is purely formal: every theorem is provable from the definition of operational semantics and the standard notion of relation; no claim is made about which systems should be compared, only about how the comparison can be made rigorous when undertaken.

Keywords: operational bisimulation, equation identity, coinduction, partition refinement, labeled transition systems, cross-framework verification, compositional reasoning, operational semantics.

Contents

I	Preliminaries	2
1	Introduction	2
1.1	The cross-framework problem . . .	2
1.2	Why bisimulation	3
1.3	Overview of main results	3
2	Labeled Transition Systems	3
2.1	Definition	3
2.2	Configurations as computation states	3
2.3	Observable functions	3
2.4	Trajectories and computations . . .	4
2.5	Equation identity	4

II Bisimulation	4	VI Discussion	9
3 Operational Bisimulation	4	12 Comparison with Other Equivalence Notions	9
3.1 Definition	4	12.1 Denotational equivalence	9
3.2 Soundness for equation identity . .	4	12.2 Trace equivalence	9
3.3 Bisimilarity is the largest bisimulation	5	12.3 Observational equivalence	9
		12.4 Extensional equivalence	9
III Coinduction and Compositionality	5	13 Limitations and Open Questions	9
4 Coinductive Reasoning	5	13.1 Infinite configuration spaces	9
4.1 The proof principle	5	13.2 Choice of observation function . . .	9
4.2 Up-to bisimulation	5	13.3 Probabilistic and quantitative extensions	9
5 Compositionality	6	13.4 Higher-order systems	10
5.1 Sequential composition	6	14 Conclusion	10
5.2 Closure under contexts	6		
IV Decidability and Algorithms	6	Part I	
6 Decidability via Partition Refinement	6	Preliminaries	
6.1 The decision problem	6	1 Introduction	
6.2 Algorithm	6	1.1 The cross-framework problem	
7 Determinism Coincidence	7	Two distinct formalisms frequently claim to express the same underlying equation. A textbook arithmetic operator may be specified by a big-step reduction relation in one treatment and a small-step relation in another; a queue may be implemented as a linked list in one library and as a paired deque in another; an algorithm may be presented in recursive form in one paper and in iterative form in another. In each case the claim is that the two presentations are interchangeable: any input that one accepts produces the same output as it would on the other.	
7.1 Deterministic systems	7	How is such a claim to be verified? The most direct approach—computing both outputs on every input and checking equality—is rarely feasible because the input domain is typically infinite. A second approach—producing a single mathematical proof of input-output equivalence by induction over the input structure—is feasible in principle but suffers from being tied to a particular structural recursion: a proof that works for two arithmetic semantics does not, without adaptation, transfer to a proof that two queues are equivalent.	
V Worked Examples	8	This paper proposes a third approach, drawn from the operational-semantics tradition of pro-	
8 Two Operational Semantics for Arithmetic	8		
8.1 Big-step semantics	8		
8.2 Small-step semantics	8		
8.3 Bisimulation	8		
9 Two Implementations of a Queue	8		
9.1 Linked-list queue	8		
9.2 Paired-stack queue	8		
9.3 Bisimulation	8		
10 Two Computations of the Factorial	8		
10.1 Recursive factorial	8		
10.2 Iterative factorial	8		
10.3 Bisimulation	8		
11 Two Finite-State Automata for the Same Language	9		

cess calculi (Milner, 1980; Park, 1981; Milner, 1989). The approach is to define a relation between the configurations (intermediate states) of the two systems, exhibit that the relation is preserved under transitions, and check that related configurations produce the same observable outputs at termination. The relation is called an *operational bisimulation*, and its existence is sufficient to establish that the two systems express the same equation.

1.2 Why bisimulation

The bisimulation methodology has the following structural advantages over the direct-equivalence and induction-on-inputs approaches.

Coinductive reasoning. A bisimulation is a relation; once the relation is exhibited and shown to be closed under transitions, equivalence on all inputs follows automatically. There is no need to reason inductively over input structure.

Modularity. Bisimulations between sub-components compose into bisimulations between composites (Theorem 5.2). A proof of equivalence for an arithmetic expression evaluator can be built from proofs for its constituent operations.

Decidability for finite systems. When the configuration spaces are finite, bisimilarity is decidable in time $\mathcal{O}((m+n)\log n)$ via the partition-refinement algorithm of Paige and Tarjan (Paige and Tarjan, 1987).

Domain-agnosticism. The methodology applies uniformly to any two systems whose operational semantics are expressed as labeled transition systems. The semantics may model arithmetic, data structures, concurrent processes, or other categories of computation; the bisimulation methodology does not require domain-specific reasoning.

1.3 Overview of main results

We establish the following:

1. **Theorem 3.3** (Soundness): if there exists a bisimulation between the initial configurations of two systems, then the two systems compute the same equation.
2. **Theorem 3.4** (Largest Bisimulation): the union of all bisimulations is itself a bisimulation; bisimilarity is therefore the largest equivalence preserving observations.
3. **Theorem 5.2** (Compositionality): bisimulation is preserved under sequential composition,

parallel composition, and contextual placement.

4. **Theorem 4.3** (Up-To Coinduction): bisimulation admits a modular proof principle in which the relation may be closed up to bisimulation itself.
5. **Theorem 7.2** (Determinism Coincidence): for deterministic systems, bisimulation coincides with extensional equivalence on inputs.
6. **Theorem 6.2** (Decidability): bisimilarity between finite labeled transition systems is decidable in time $\mathcal{O}((m+n)\log n)$.

2 Labeled Transition Systems

2.1 Definition

Definition 2.1 (Labeled Transition System). A labeled transition system (*LTS*) is a quadruple

$$\mathcal{S} = \langle \mathcal{C}, \mathcal{L}, \rightarrow, \mathbf{l} \rangle$$

where $\mathcal{C}$ is a set of configurations, $\mathcal{L}$ is a set of labels, $\rightarrow \subseteq \mathcal{C} \times \mathcal{L} \times \mathcal{C}$ is a transition relation, and $\mathbf{l} \subseteq \mathcal{C}$ is a designated set of initial configurations. We write $c \xrightarrow{a} c'$ for $(c, a, c') \in \rightarrow$, and we say configuration c steps to c' with label a .

Remark 2.2. The general formulation of *LTSes* is due to Plotkin (1981) in the context of structural operational semantics, and is used as the foundational vocabulary of process algebra in Milner (1989); Hoare (1985); Bergstra and Klop (1984). We adopt the formulation here in its standard form.

2.2 Configurations as computation states

For our purposes, configurations represent intermediate computational states. The initial configurations represent inputs; the transitions represent computation steps; terminal configurations (those with no outgoing transitions) represent results.

Definition 2.3 (Terminal Configuration). A configuration $c \in \mathcal{C}$ is terminal if there is no $a \in \mathcal{L}$ and $c' \in \mathcal{C}$ with $c \xrightarrow{a} c'$. The set of terminal configurations is denoted $\mathbf{T} \subseteq \mathcal{C}$.

2.3 Observable functions

To talk about “equation identity” we must specify what we observe about a configuration. We attach to each LTS an observation function.

Definition 2.4 (Observation Function). An observation function for an LTS $\mathcal{S}$ is a partial function $\text{obs} : \mathcal{C} \rightarrow \mathcal{O}$, where $\mathcal{O}$ is a designated set of outputs. The function is defined exactly on the terminal configurations: $\text{dom}(\text{obs}) = \mathsf{T}$.

Remark 2.5. Restricting obs to terminal configurations is a deliberate methodological choice. It encodes that the equation we wish to verify is an input-output relation, not a behaviour-along-the-way relation. Two systems that produce the same final output via different intermediate steps are equation-equivalent under this notion.

2.4 Trajectories and computations

Definition 2.6 (Trajectory). A trajectory of $\mathcal{S}$ from configuration c_0 is a (finite or infinite) sequence $c_0 \xrightarrow{a_1} c_1 \xrightarrow{a_2} c_2 \xrightarrow{a_3} \dots$. A trajectory terminates if it is finite and ends in a configuration $c_n \in \mathsf{T}$.

Definition 2.7 (Computation). A computation of $\mathcal{S}$ from c_0 is a terminating trajectory. The result of a computation is $\text{obs}(c_n)$, where c_n is the terminal configuration of the computation.

2.5 Equation identity

Definition 2.8 (Equation Identity). Two LTSes $\mathcal{S}_A$ and $\mathcal{S}_B$ with shared label alphabet, output set $\mathcal{O}$, observation functions $\text{obs}_A, \text{obs}_B$, and bijection $\phi : \mathsf{I}_A \rightarrow \mathsf{I}_B$ between their initial configurations are said to compute the same equation if for every $c_0 \in \mathsf{I}_A$:

1. every computation of $\mathcal{S}_A$ from c_0 has a corresponding computation of $\mathcal{S}_B$ from $\phi(c_0)$ with the same result;
2. every computation of $\mathcal{S}_B$ from $\phi(c_0)$ has a corresponding computation of $\mathcal{S}_A$ from c_0 with the same result.

Remark 2.9. Definition 2.8 is symmetric: the two systems are required to produce the same outputs in both directions. When both systems are deterministic, the definition simplifies to “for every initial c_0 , the unique computation of $\mathcal{S}_A$ produces the same result as the unique computation of $\mathcal{S}_B$ from $\phi(c_0)$ ”. The general formulation accommodates non-deterministic systems by requiring matching computations to exist in both directions.

Part II

Bisimulation

3 Operational Bisimulation

3.1 Definition

Definition 3.1 (Bisimulation). Let $\mathcal{S}_A = \langle \mathcal{C}_A, \mathcal{L}, \rightarrow_A, \mathsf{I}_A \rangle$ and $\mathcal{S}_B = \langle \mathcal{C}_B, \mathcal{L}, \rightarrow_B, \mathsf{I}_B \rangle$ be LTSes with observation functions $\text{obs}_A, \text{obs}_B$ into a common output set $\mathcal{O}$. A relation $R \subseteq \mathcal{C}_A \times \mathcal{C}_B$ is a bisimulation between $\mathcal{S}_A$ and $\mathcal{S}_B$ if for every $(c, d) \in R$:

- (B1) if $c \xrightarrow{a}_A c'$, then there exists $d' \in \mathcal{C}_B$ with $d \xrightarrow{a}_B d'$ and $(c', d') \in R$;
- (B2) if $d \xrightarrow{a}_B d'$, then there exists $c' \in \mathcal{C}_A$ with $c \xrightarrow{a}_A c'$ and $(c', d') \in R$;
- (B3) if $c \in \mathsf{T}_A$, then $d \in \mathsf{T}_B$ and $\text{obs}_A(c) = \text{obs}_B(d)$;
- (B4) if $d \in \mathsf{T}_B$, then $c \in \mathsf{T}_A$ and $\text{obs}_A(c) = \text{obs}_B(d)$.

Two configurations c, d are bisimilar (written $c \sim d$) if there exists a bisimulation containing (c, d) .

Remark 3.2. The first two clauses are the standard transition-matching conditions of [Park \(1981\)](#); [Milner \(1989\)](#). The third and fourth clauses are the termination conditions that connect bisimulation to equation identity: bisimilar terminal configurations must produce the same output, and bisimilar transitions must lead to bisimilar terminations.

3.2 Soundness for equation identity

Theorem 3.3 (Soundness). Let $\phi : \mathsf{I}_A \rightarrow \mathsf{I}_B$ be a bijection between initial configurations. If for every $c_0 \in \mathsf{I}_A$ we have $c_0 \sim \phi(c_0)$, then $\mathcal{S}_A$ and $\mathcal{S}_B$ compute the same equation.

Proof. Fix $c_0 \in \mathsf{I}_A$. We verify both directions of Definition 2.8.

Direction 1. Let $c_0 \xrightarrow{a_1}_A c_1 \xrightarrow{a_2}_A c_2 \xrightarrow{a_3}_A \dots \xrightarrow{a_n}_A c_n$ be a computation of $\mathcal{S}_A$ from c_0 . We construct, by induction on i , a sequence $d_0 = \phi(c_0), d_1, d_2, \dots, d_n$ with $c_i \sim d_i$ and $d_{i-1} \xrightarrow{a_i}_B d_i$ for each i .

Base. $c_0 \sim d_0$ by hypothesis.

Step. Given $c_{i-1} \sim d_{i-1}$ and $c_{i-1} \xrightarrow{a_i}_A c_i$, apply clause (B1) of Definition 3.1 to obtain d_i with $d_{i-1} \xrightarrow{a_i}_B d_i$ and $c_i \sim d_i$.

At $i = n$, the configuration c_n is terminal. By clause (B3), d_n is terminal and $\text{obs}_A(c_n) =$

$\text{obs}_B(d_n)$. The constructed sequence $d_0 \xrightarrow{a_1}_B \dots \xrightarrow{a_n}_B d_n$ is therefore a computation of $\mathcal{S}_B$ from $\phi(c_0)$ with the same result.

Direction 2. Symmetric, using clauses (B2) and (B4).

Both directions hold for every $c_0 \in \mathcal{I}_A$, so the systems compute the same equation. $\square$ $\square$

3.3 Bisimilarity is the largest bisimulation

Theorem 3.4 (Largest Bisimulation). *The bisimilarity relation $\sim$ is itself a bisimulation. Moreover, every bisimulation R satisfies $R \subseteq \sim$.*

Proof. $\sim$ is a bisimulation. Let $(c, d) \in \sim$. By Definition 3.1, there exists some bisimulation R with $(c, d) \in R$. If $c \xrightarrow{a}_A c'$, then R provides d' with $d \xrightarrow{a}_B d'$ and $(c', d') \in R \subseteq \sim$. The other clauses follow analogously. Hence $\sim$ satisfies all four clauses of Definition 3.1.

Maximality. If R is a bisimulation with $(c, d) \in R$, then $c \sim d$ by definition, so $R \subseteq \sim$. $\square$ $\square$

Corollary 3.5 (Equivalence Relation). *The bisimilarity relation $\sim$ is reflexive, symmetric, and transitive on a single LTS (identifying $\mathcal{S}_A = \mathcal{S}_B$).*

Proof. *Reflexivity.* The identity relation $\{(c, c) : c \in \mathcal{C}\}$ is a bisimulation: $c \xrightarrow{a}_A c'$ matches $c \xrightarrow{a}_A c'$ with $(c', c') \in \text{Id}$, and $\text{obs}(c) = \text{obs}(c)$ trivially.

Symmetry. If R is a bisimulation, so is $R^{-1} = \{(d, c) : (c, d) \in R\}$, by exchanging clauses (B1)/(B2) and (B3)/(B4).

Transitivity. If R_1, R_2 are bisimulations, so is their relational composition $R_1 \circ R_2 = \{(c, e) : \exists d. (c, d) \in R_1 \wedge (d, e) \in R_2\}$. The matching properties compose transitively. $\square$ $\square$

Part III

Coinduction and Compositionality

4 Coinductive Reasoning

4.1 The proof principle

The largest-bisimulation theorem yields a powerful proof principle. To establish $c \sim d$, it suffices

to exhibit a single relation R with $(c, d) \in R$ that satisfies the bisimulation clauses. The relation need not be the largest; any bisimulation suffices, and any bisimulation is contained in $\sim$.

Proposition 4.1 (Coinductive Proof Principle). *To prove $c \sim d$, it suffices to construct any relation R with $(c, d) \in R$ and verify that R satisfies the four bisimulation clauses (B1)–(B4) for every pair in R .*

Proof. By Definition 3.1, R being a bisimulation containing (c, d) is the definition of $c \sim d$. $\square$ $\square$

4.2 Up-to bisimulation

A frequent enhancement of the proof principle allows the proof relation to be closed up to bisimulation itself, eliminating extraneous side conditions.

Definition 4.2 (Bisimulation Up To). *A relation $R \subseteq \mathcal{C}_A \times \mathcal{C}_B$ is a bisimulation up to $\sim$ if for every $(c, d) \in R$:*

- if $c \xrightarrow{a}_A c'$, there exist c'', d', d'' with $c' \sim c''$, $d \xrightarrow{a}_B d'$, $d' \sim d''$, and $(c'', d'') \in R$;
- the symmetric clause for $d \xrightarrow{a}_B d'$;
- the termination clauses (B3) and (B4) of Definition 3.1.

Theorem 4.3 (Up-To Coinduction). *If R is a bisimulation up to $\sim$ and $(c, d) \in R$, then $c \sim d$.*

Proof. Define $R' = \sim \circ R \circ \sim$, the composition $\sim$ -then- R -then- $\sim$. We show R' is a (standard) bisimulation containing (c, d) .

Membership. By reflexivity (Corollary 3.5), $(c, c) \in \sim$ and $(d, d) \in \sim$, so $(c, d) \in \sim \circ R \circ \sim = R'$ when $(c, d) \in R$.

Bisimulation conditions. Let $(c_1, d_1) \in R'$. By construction, there exist c_2, d_2 with $c_1 \sim c_2$, $(c_2, d_2) \in R$, and $d_2 \sim d_1$.

Suppose $c_1 \xrightarrow{a}_A c'_1$. By the bisimulation property of $\sim$, there exists c'_2 with $c_2 \xrightarrow{a}_A c'_2$ and $c'_1 \sim c'_2$. By the up-to bisimulation property of R , there exist c_3, d_3, d'_2 with $c'_2 \sim c_3$, $d_2 \xrightarrow{a}_B d'_2$, $d'_2 \sim d_3$, and $(c_3, d_3) \in R$. By the bisimulation property of $\sim$ applied to $d_2 \sim d_1$ and $d_2 \xrightarrow{a}_B d'_2$, there exists d'_1 with $d_1 \xrightarrow{a}_B d'_1$ and $d'_2 \sim d'_1$.

Putting these together: $c'_1 \sim c'_2 \sim c_3$, $(c_3, d_3) \in R$, and $d_3 \sim d'_2 \sim d'_1$. By transitivity of $\sim$, $c'_1 \sim c_3$ and $d_3 \sim d'_1$, so $(c'_1, d'_1) \in \sim \circ R \circ \sim = R'$.

The symmetric clause and the termination clauses follow similarly. Hence R' is a bisimulation containing (c, d) , so $c \sim d$. $\square$ $\square$

Remark 4.4. *Up-to bisimulation is due to Milner and Sangiorgi (1992) in the context of π -calculus, and its theory is systematically developed in Sangiorgi and Rutten (2011). The principle is widely used because it allows proofs to abstract away from configuration structure that is bisimilar but not literally equal.*

5 Compositionality

5.1 Sequential composition

Definition 5.1 (Sequential Composition of LTSes). *Let $\mathcal{S}_1$ and $\mathcal{S}_2$ be LTSes such that the output set $\mathcal{O}_1$ of $\mathcal{S}_1$ coincides with the initial-configuration domain of $\mathcal{S}_2$. The sequential composition $\mathcal{S}_1; \mathcal{S}_2$ is the LTS whose configurations are pairs $(c_1, c_2) \in \mathcal{C}_1 \uplus \mathcal{C}_2$, whose transitions are those of $\mathcal{S}_1$ until c_1 becomes terminal at which point the result $\text{obs}_1(c_1)$ becomes the initial configuration of $\mathcal{S}_2$, and whose observation function is obs_2 .*

Theorem 5.2 (Compositionality). *If $\mathcal{S}_A \sim \mathcal{S}_B$ (componentwise) and $\mathcal{S}'_A \sim \mathcal{S}'_B$, then:*

1. $\mathcal{S}_A; \mathcal{S}'_A \sim \mathcal{S}_B; \mathcal{S}'_B$ under sequential composition.
2. $\mathcal{S}_A \parallel \mathcal{S}'_A \sim \mathcal{S}_B \parallel \mathcal{S}'_B$ under parallel composition, where $\parallel$ denotes any parallel-composition operator whose semantics respects bisimulation of components.

Proof. We give the sequential case in detail; the parallel case follows analogously.

Let R_1 be a bisimulation witnessing $\mathcal{S}_A \sim \mathcal{S}_B$, and let R_2 witness $\mathcal{S}'_A \sim \mathcal{S}'_B$. Define

$$R = \{((c_A, c'_A), (c_B, c'_B)) : (c_A, c_B) \in R_1 \text{ and } (c'_A, c'_B) \in R_2\}.$$

We show R is a bisimulation for $\mathcal{S}_A; \mathcal{S}'_A$ and $\mathcal{S}_B; \mathcal{S}'_B$.

A transition in $\mathcal{S}_A; \mathcal{S}'_A$ from (c_A, c'_A) comes either from $\mathcal{S}_A$ (while c_A is non-terminal) or from $\mathcal{S}'_A$ (after c_A becomes terminal and its output initialises c'_A). In the first case, clause (B1) of R_1 matches the transition in $\mathcal{S}_B$, preserving membership in R . In the second case, clause (B3) of R_1 ensures that c_B becomes terminal at the same step with $\text{obs}_A(c_A) = \text{obs}_B(c_B)$, so both compositions initialise their continuations to the same value; clause (B1) of R_2 then matches subsequent transitions.

The termination clauses follow similarly. Hence R is a bisimulation, and the sequential composition preserves bisimilarity. $\square$ $\square$

5.2 Closure under contexts

Corollary 5.3 (Contextual Closure). *If $\mathcal{S}_A \sim \mathcal{S}_B$ and $C[\cdot]$ is any context built from sequential and parallel composition (and other bisimulation-respecting operators), then $C[\mathcal{S}_A] \sim C[\mathcal{S}_B]$.*

Proof. By induction on the structure of $C[\cdot]$, applying Theorem 5.2 at each step. $\square$ $\square$

Remark 5.4. *Corollary 5.3 is the property called congruence in process algebra (Milner, 1989). It is what makes bisimulation a usable equivalence: equivalent components remain equivalent when plugged into any context.*

Part IV

Decidability and Algorithms

6 Decidability via Partition Refinement

6.1 The decision problem

Definition 6.1 (Bisimilarity Decision Problem). *Given two finite LTSes $\mathcal{S}_A, \mathcal{S}_B$ and a pair $(c_0, d_0) \in \mathcal{I}_A \times \mathcal{I}_B$, the bisimilarity decision problem is to decide whether $c_0 \sim d_0$.*

6.2 Algorithm

The standard procedure for deciding bisimilarity is partition refinement, originally due to Kanelakis and Smolka (1990) and improved by Paige and Tarjan (1987) to near-linear complexity.

Algorithm 1 Partition-Refinement Bisimilarity

```

1: function IsBISIM( $\mathcal{S}_A, \mathcal{S}_B, c_0, d_0$ )
2:   Let  $\mathcal{C} = \mathcal{C}_A \uplus \mathcal{C}_B$ , the disjoint union
3:   Let  $\Pi$  be the partition of  $\mathcal{C}$  where two
   configurations  $c, c'$ 
4:     share a block iff  $\text{obs}(c) = \text{obs}(c')$  (ter-
   minal cases) or both
5:     are non-terminal
6:   repeat
7:      $\Pi' \leftarrow \Pi$ 
8:     for each block  $B \in \Pi$  and each label
    $a \in \mathcal{L}$  do
9:       Refine  $B$  by post-image: two con-
   figurations  $c, c'$  remain
10:      in the same sub-block iff their
    $a$ -successors fall
11:      in the same blocks of  $\Pi$ 
12:     end for
13:   until  $\Pi$  is stable:  $\Pi = \Pi'$ 
14:   return whether  $c_0$  and  $d_0$  are in the same
   block of  $\Pi$ 
15: end function

```

Theorem 6.2 (Decidability). *Algorithm 1 decides bisimilarity between two finite LTSes in time $\mathcal{O}((m+n)\log n)$, where $n = |\mathcal{C}|$ and $m = |\rightarrow|$.*

Proof. Correctness. The initial partition identifies configurations with the same observable behaviour at termination (the observation function) and treats all non-terminal configurations as initially indistinguishable. Each refinement step splits any block whose configurations have transitions to different blocks. Upon stabilisation, two configurations are in the same block iff they have indistinguishable behaviour under all transitions and observations, which is precisely bisimilarity.

Complexity. The Paige–Tarjan algorithm (Paige and Tarjan, 1987) performs $\mathcal{O}((m+n)\log n)$ work by maintaining auxiliary data structures that bound the per-block refinement cost. Each block of size k contributes $\mathcal{O}(k \log k)$ work over the algorithm’s run, and the sum of $k \log k$ over disjoint blocks is bounded by $n \log n$. The transitions add a contribution proportional to m .

Termination. The partition refines monotonically (blocks only split, never merge), and there are at most n blocks. Hence the loop terminates after at most n iterations. $\square$ $\square$

Remark 6.3. *The Paige–Tarjan bound is essentially optimal for the bisimilarity problem on*

labeled transition systems (Paige and Tarjan, 1987). For practical implementations, the $\log n$ factor can be reduced for restricted LTS classes; see Kanellakis and Smolka (1990) for a comprehensive treatment.

7 Determinism Coincidence

7.1 Deterministic systems

Definition 7.1 (Determinism). *An LTS $\mathcal{S}$ is deterministic if for every configuration c and label a , there is at most one c' with $c \xrightarrow{a} c'$. Equivalently, the transition relation is a partial function $\mathcal{C} \times \mathcal{L} \rightarrow \mathcal{C}$.*

Theorem 7.2 (Determinism Coincidence). *Let $\mathcal{S}_A, \mathcal{S}_B$ be deterministic LTSes with a label-preserving bijection between their transition structures. Then $c_0 \sim d_0$ if and only if the unique computation of $\mathcal{S}_A$ from c_0 produces the same output as the unique computation of $\mathcal{S}_B$ from d_0 .*

Proof. ($\Rightarrow$) If $c_0 \sim d_0$, by Theorem 3.3 the two systems compute the same equation, so in particular they produce the same output.

($\Leftarrow$) Suppose the unique computations from c_0 and d_0 have the same result. Define R as the set of pairs (c, d) such that c is reached at step i of $\mathcal{S}_A$ ’s unique computation from c_0 and d is reached at step i of $\mathcal{S}_B$ ’s unique computation from d_0 . By determinism, the transitions are pairwise matched (each step has a unique successor). At termination, the outputs agree by hypothesis. Hence R is a bisimulation containing (c_0, d_0) . $\square$ $\square$

Corollary 7.3 (Equation Identity for Deterministic Systems). *For deterministic systems, bisimulation methodology and direct input-output comparison are interchangeable: the same equation-identity claims are provable by either approach.*

Proof. Direct from Theorem 7.2. $\square$ $\square$

Remark 7.4. *Corollary 7.3 is not trivial: it says that in the deterministic case, the additional power of bisimulation (the ability to reason about behaviour along the way) reduces to the simpler extensional comparison. The advantage of the bisimulation framework in this case is not increased power but increased modularity: a bisimulation is a single relation, while extensional comparison requires reasoning about each input separately.*

Part V

Worked Examples

8 Two Operational Semantics for Arithmetic

8.1 Big-step semantics

Consider arithmetic expressions over integers with addition and multiplication:

$$e ::= n \mid e_1 + e_2 \mid e_1 \times e_2.$$

The big-step semantics gives a relation $e \Downarrow v$ assigning each expression to its value:

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 + e_2 \Downarrow v_1 + v_2}, \quad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 \times e_2 \Downarrow v_1 \times v_2}.$$

The configurations are pairs $(e, \star)$ where $\star$ denotes “unfinished”; the transition is $(e, \star) \xrightarrow{e \Downarrow v} (v, \text{done})$. The observation function returns v on terminal configurations.

8.2 Small-step semantics

The small-step semantics reduces expressions step-wise via congruence rules:

$$\begin{aligned} e_1 + e_2 &\rightarrow e'_1 + e_2 \text{ if } e_1 \rightarrow e'_1, \\ e_1 + e_2 &\rightarrow e_1 + e'_2 \text{ if } e_1 \text{ is a value and } e_2 \rightarrow e'_2, \\ n_1 + n_2 &\rightarrow n_1 + n_2 \text{ as a primitive numeric operation,} \end{aligned}$$

and similarly for multiplication. Configurations are expressions; transitions reduce; terminal configurations are integer literals.

8.3 Bisimulation

Define $R = \{((e, \star), e) : e \text{ is a closed expression}\} \cup \{((n, \text{done}), n) : n \in \mathbb{Z}\}$. We verify R is a bisimulation.

For $((e, \star), e) \in R$ with e non-terminal in both systems, the big-step transition derives $e \Downarrow v$ and reaches $((v, \text{done}), v)$. The small-step semantics reaches v via a sequence of intermediate reductions; the final state is v . Up to bisimulation (Theorem 4.3), the intermediate states are matched. The termination outputs agree: both return v .

The bisimulation establishes that the two arithmetic semantics compute the same equation.

9 Two Implementations of a Queue

9.1 Linked-list queue

A queue implemented as a singly linked list: enqueue appends to the tail in $\mathcal{O}(1)$, dequeue removes from the head in $\mathcal{O}(1)$, both with tail-pointer maintenance.

9.2 Paired-stack queue

A queue implemented as two stacks $(S_{\text{front}}, S_{\text{back}})$ (Hood and Melville, 1981). Enqueue pushes to S_{back} . Dequeue pops from S_{front} ; if S_{front} is empty, the contents of S_{back} are reversed onto S_{front} first.

9.3 Bisimulation

Two queue states are related when their visible FIFO order is the same. Formally:

$$R = \{(L, (S_f, S_b)) : L = S_f \widehat{\ } \text{rev}(S_b)\},$$

where $\widehat{\ }$ is concatenation. Enqueue in the linked list appends x to L ; enqueue in the paired stack pushes x to S_b , yielding $S_f \widehat{\ } \text{rev}(x \cdot S_b) = S_f \widehat{\ } \text{rev}(S_b) \widehat{\ } [x] = L \widehat{\ } [x]$. Dequeue similarly preserves the relation, including the rebalancing case when S_f becomes empty (the reversal makes the new S_f equal $\text{rev}(S_b)$, and the next dequeue removes the head). Terminal configurations (queue is empty) coincide. The bisimulation establishes equation identity.

10 Two Computations of the Factorial

10.1 Recursive factorial

The recursive definition: $\text{fact}(0) = 1$, $\text{fact}(n) = n \cdot \text{fact}(n - 1)$ for $n > 0$. Implemented as a stack of recursive calls.

10.2 Iterative factorial

The iterative definition: maintain (i, acc) starting at $(n, 1)$; on each step, $\text{acc} \leftarrow i \cdot \text{acc}$ and $i \leftarrow i - 1$; terminate when $i = 0$.

10.3 Bisimulation

Relate the recursive call stack of depth k from initial argument n to the iterative state $(n - k, \prod_{j=n-k+1}^n j)$. The transitions match: a recursive return multiplies the running product by the popped value and decreases the stack depth; the iterative step does the equivalent multiplication and decrement. Both terminate at $fact(n)$. The bisimulation establishes equation identity.

11 Two Finite-State Automata for the Same Language

Two deterministic finite automata A_1, A_2 accept the same regular language iff there exists a bisimulation between them (Hopcroft and Ullman, 1979). The classical minimal-DFA construction (Hopcroft, 1971) computes the bisimilarity partition and produces the minimal automaton. Algorithm 1 applied to A_1, A_2 (treating accepting states as terminal with output $\top$ and rejecting states as terminal with output $\perp$) decides language equivalence in $\mathcal{O}((m+n)\log n)$.

Part VI Discussion

12 Comparison with Other Equivalence Notions

12.1 Denotational equivalence

Denotational semantics (Scott and Strachey, 1971; Winskel, 1993) assigns each program a mathematical object (typically a function on a domain); two programs are denotationally equivalent if they receive the same denotation. Bisimulation differs by reasoning at the operational level: it asks whether the two computations match step-by-step, not whether they share a denotation. The two notions coincide for well-behaved denotational models but diverge in the presence of non-determinism or infinite computation.

12.2 Trace equivalence

Trace equivalence (Hoare, 1985) asks whether the two systems produce the same set of finite trajectories. Trace equivalence is strictly weaker

than bisimulation: bisimilar systems are trace-equivalent, but trace-equivalent systems may distinguish under nesting in contexts that observe branching structure (van Glabbeek, 2001).

12.3 Observational equivalence

The observational equivalence of Morris (1968) (and its operational refinement in Plotkin, 1977) considers two programs equivalent if no context can distinguish them by terminating behaviour. Bisimulation implies observational equivalence on systems with sufficiently rich observation functions; the converse holds for many calculi but not all.

12.4 Extensional equivalence

For deterministic systems, extensional equivalence (same outputs on same inputs) coincides with bisimulation by Theorem 7.2. For non-deterministic systems, extensional equivalence does not in general imply bisimulation: two systems may produce the same set of possible outputs by different branching structures.

13 Limitations and Open Questions

13.1 Infinite configuration spaces

Algorithm 1 requires finite LTSes. Systems with infinite configuration spaces (e.g. Turing machines, unbounded counters) require alternative decision procedures, when one exists; bisimilarity is undecidable in general for Turing-equivalent systems.

13.2 Choice of observation function

The methodology depends on a chosen observation function. Different choices yield different equivalences; what counts as “the same equation” depends on what is observable. We have left `obs` as a parameter of the framework, but in practice its choice is application-specific.

13.3 Probabilistic and quantitative extensions

Bisimulation has well-developed extensions to probabilistic systems (Larsen and Skou, 1991) and to systems with quantitative observations (timed

automata, weighted transitions). The methodology developed here generalises straightforwardly to these settings by replacing equality with the appropriate metric, but we do not pursue the generalisations here.

13.4 Higher-order systems

For higher-order languages where values themselves are computations (e.g. functional programs over λ -terms), bisimulation requires careful treatment of the function space (Abramsky, 1990; Gordon, 1995). Our development assumes first-order observations; the higher-order case is an active area of research and outside our scope.

14 Conclusion

We have developed a methodology for proving that two formalisms with distinct operational semantics compute the same equation, based on operational bisimulation between their labeled transition systems. The methodology rests on six results: the Soundness Theorem (Theorem 3.3) connecting bisimulation to equation identity; the Largest-Bisimulation Theorem (Theorem 3.4) establishing the canonical equivalence; the Compositionality Theorem (Theorem 5.2) ensuring bisimulation composes under sequential and parallel composition; the Up-To Theorem (Theorem 4.3) providing a modular proof principle; the Determinism Coincidence Theorem (Theorem 7.2) showing bisimulation reduces to extensional equivalence in the deterministic case; and the Decidability Theorem (Theorem 6.2) giving $O((m+n)\log n)$ algorithmic verification.

The methodology is purely formal. It does not commit to a particular family of systems or a particular notion of correctness; it offers, instead, a uniform procedure for moving from a claim that two systems express the same equation to a proof that they do. The four worked examples—arithmetic semantics, queue implementations, factorial computations, and finite-state automata—illustrate the procedure across distinct application domains.

The framework is foundational. It supports the analysis of any cross-framework equation-identity claim whose two formalisms admit operational semantics. The choices of observation function, label alphabet, and configuration space

are application-specific; the methodology of relating them by bisimulation is uniform.

References

- Samson Abramsky. The lazy lambda calculus. *Research Topics in Functional Programming*, pages 65–116, 1990.
- Jan A. Bergstra and Jan Willem Klop. Process algebra for synchronous communication. *Information and Control*, 60(1-3):109–137, 1984.
- Andrew D. Gordon. Bisimilarity as a theory of functional programming. *Electronic Notes in Theoretical Computer Science*, 1:232–252, 1995.
- C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall, 1985.
- Robert Hood and Robert Melville. Real-time queue operations in pure LISP. *Information Processing Letters*, 13(2):50–54, 1981.
- John E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In Zvi Kohavi and Azaria Paz, editors, *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.
- John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 1979.
- Paris C. Kanellakis and Scott A. Smolka. CCS expressions, finite state processes, and three problems of equivalence. *Information and Computation*, 86(1):43–68, 1990.
- Kim G. Larsen and Arne Skou. Bisimulation through probabilistic testing. *Information and Computation*, 94(1):1–28, 1991.
- Robin Milner. *A Calculus of Communicating Systems*, volume 92 of *Lecture Notes in Computer Science*. Springer, 1980.
- Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- Robin Milner and Davide Sangiorgi. *Barbed Bisimulation*, volume 623 of *Lecture Notes in Computer Science*. Springer, 1992.
- James H. Morris. *Lambda-Calculus Models of Programming Languages*. PhD thesis, MIT, 1968.

- Robert Paige and Robert E. Tarjan. Three partition refinement algorithms. *SIAM Journal on Computing*, 16(6):973–989, 1987.
- David Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science, 5th GI-Conference*, volume 104 of *Lecture Notes in Computer Science*, pages 167–183. Springer, 1981.
- Gordon D. Plotkin. LCF considered as a programming language. *Theoretical Computer Science*, 5(3):223–255, 1977.
- Gordon D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, 1981.
- Davide Sangiorgi and Jan Rutten. *Advanced Topics in Bisimulation and Coinduction*. Cambridge University Press, 2011.
- Dana S. Scott and Christopher Strachey. Toward a mathematical semantics for computer languages. *Proceedings of the Symposium on Computers and Automata*, pages 19–46, 1971.
- Rob J. van Glabbeek. The linear time – branching time spectrum I: The semantics of concrete, sequential processes. *Handbook of Process Algebra*, pages 3–99, 2001.
- Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.